

Messages de vitesse en vol

Javions – étape 6

1. Introduction

Le but principal de cette sixième étape est d'écrire la classe représentant le dernier type de message ADS-B traité dans ce projet, qui permet aux aéronefs de communiquer leur vitesse de déplacement lorsqu'ils sont en vol.

Notez que cette étape devra être rendue deux fois :

- 1. pour le rendu testé habituel (délai : **31/3 18h00**),
- 2. pour le rendu intermédiaire (délai : **7/4 18h00**).

Le deuxième de ces rendus sera corrigé par lecture du code de vos étapes 1 à 6, et il vous faudra donc soigner sa qualité et sa documentation. Il est fortement conseillé de lire [notre guide à ce sujet](#).

2. Concepts

2.1. Messages de vitesse en vol

Les messages de **vitesse en vol** (*airborne velocity*) permettent aux aéronefs de communiquer leur vitesse et direction de déplacement lorsqu'ils sont en vol.

Ces messages ont un code de type valant 19, stocké pour mémoire dans les 5 bits de poids fort de l'attribut ME du message, qui en fait 56. Les parties utiles à ce projet des 51 bits restants sont décrites dans la table ci-dessous.

Nom	Début	Bits	Contenu
ST	48	3	Sous-type
IC	47	1	<i>inutilisé dans ce projet</i>
IFR	46	1	<i>inutilisé dans ce projet</i>
NUC	43	3	<i>inutilisé dans ce projet</i>
	21	22	<i>dépend du sous-type</i>
VR	10	11	<i>inutilisé dans ce projet</i>
	8	2	<i>réservé</i>
dALT	0	8	<i>inutilisé dans ce projet</i>

L'attribut ST donne le sous-type du message, qui permet de savoir comment interpréter

les 22 bits commençant au bit 21. Dans un message valide, interprété comme un entier non signé, le sous-type – qu’il ne faut pas confondre avec le code de type – ne peut valoir que 1, 2, 3 ou 4.

La raison pour laquelle il existe 4 sous-types différents est d’une part que la vitesse peut être mesurée dans deux unités différentes, en fonction de si l’aéronef est en vol subsonique ou supersonique, et d’autre part que la vitesse et la direction de déplacement peuvent être mesurées de deux manières différentes, comme décrit ci-après.

2.1.1. Mesure de la vitesse et de la direction de déplacement

Une première manière de mesurer la vitesse d’un aéronef consiste à la mesurer par rapport au sol, pour obtenir ce que l’on nomme la **vitesse sol** (*ground speed*). Cela revient à mesurer la vitesse de l’ombre que l’aéronef projette sur le sol, en admettant que le soleil se trouve exactement au-dessus de lui.

Une seconde manière de mesurer la vitesse consiste à le faire par rapport à la masse d’air dans laquelle se trouve l’aéronef, par exemple en lui attachant un anémomètre, pour obtenir ce que l’on nomme la **vitesse air** (*air speed*).

Ces deux manières de mesurer la vitesse d’un aéronef diffèrent, car la masse d’air dans laquelle il se trouve n’est généralement pas stationnaire. Ainsi, un ballon à air chaud – qui n’a aucun moyen propre de se déplacer horizontalement – a toujours une vitesse air nulle. Par contre, sa vitesse sol ne l’est souvent pas, car il se trouve dans une masse d’air qui se déplace.

La vitesse sol est celle mesurée par les systèmes de positionnement par satellite – GPS ou autres – et est généralement celle qui est communiquée par les aéronefs, car c’est la plus utile des deux pour le trafic aérien. La vitesse air est mesurée quant à elle au moyen d’un anémomètre situé sur l’aéronef.

La direction de déplacement peut également être mesurée de différentes manières. La première consiste à déterminer – conceptuellement en tout cas – la direction dans laquelle se déplace l’ombre de l’aéronef sur le sol, que l’on nomme sa **route** (*track*). La seconde consiste à mesurer, au moyen d’une boussole, la direction dans laquelle pointe le nez de l’aéronef, que l’on nomme son **cap** (*heading*). Là aussi, ces deux types de mesure produisent généralement des résultats différents en raison du mouvement de la masse d’air dans laquelle évolue l’aéronef.

2.1.2. Déplacement par rapport au sol

Lorsqu’un message de vitesse en vol a un sous-type valant 1 ou 2, il communique la vitesse sol de l’aéronef, ainsi que sa route. Dans ce cas, le contenu des 22 bits qui dépendent du sous-type est donné par la table suivante :

Nom	Début	Bits	Contenu
-----	-------	------	---------

Dew	21	1	Sens de la composante est-ouest de la vitesse
Vew	11	10	Composante est-ouest de la vitesse (+1)
Dns	10	1	Sens de la composante nord-sud de la vitesse
Vns	0	10	Composante nord-sud de la vitesse (+1)

Chacune des paires d'attributs – Dns/Vns d'un côté et Dew/Vew de l'autre – représente une composante du vecteur vitesse. Les attributs Dns et Dew indiquent le sens de la composante, tandis que les attributs Vns et Vew indiquent sa valeur absolue *plus un*. En effet, lorsque Vns ou Vew valent 0, la vitesse est inconnue.

Lorsque Dns vaut 0, l'aéronef se dirige du sud au nord, et lorsqu'il vaut 1 il se dirige du nord au sud ; lorsque Dew vaut 0, l'aéronef se dirige de l'ouest vers l'est, et lorsqu'il vaut 1 il se dirige de l'est vers l'ouest.

Une fois les composantes du vecteur vitesse obtenues, on peut en calculer la norme pour obtenir la vitesse, et l'angle pour obtenir la direction de déplacement.

Le sous-type 1 est destiné à communiquer la vitesse des avions volant à vitesse subsonique, et le vecteur vitesse est alors exprimé en nœuds. Le sous-type 2 est destiné aux avions supersoniques, et le vecteur vitesse est alors exprimé en une unité qui ne porte pas de nom mais qui correspond à 4 nœuds.

2.1.3. Déplacement dans l'air

Lorsqu'un message de vitesse en vol a un sous-type valant 3 ou 4, il communique la vitesse air de l'aéronef, ainsi que son cap. Dans ce cas, le contenu des 22 bits qui dépendent du sous-type est donné par la table suivante :

Nom	Début	Bits	Contenu
SH	21	1	Disponibilité du cap
HDG	11	10	Cap
T	10	1	<i>inutilisé dans ce projet</i>
AS	0	10	Vitesse air

Si le bit SH vaut 1, alors l'attribut HDG contient le cap de l'aéronef, c.-à-d. la direction dans laquelle son nez pointe. En interprétant cet attribut comme un entier non signé puis en le divisant par 2^{10} , on obtient le cap exprimé en tours. L'angle ainsi obtenu est celui séparant le nord et la direction dans laquelle pointe le nez de l'aéronef, mesuré dans le sens horaire.

Ainsi, le cap d'un aéronef vaut 0° s'il pointe son nez vers le nord, 90° s'il le pointe vers l'est, 180° s'il le pointe vers le sud, et 270° s'il le pointe vers l'ouest.

Quand le sous-type du message vaut 3, la vitesse air contenue dans l'attribut AS est exprimée en nœuds, alors qu'elle est exprimée en « 4 nœuds » si le sous-type vaut 4.

2.1.4. Unification

Même si les deux concepts de vitesse et de direction de déplacement décrits ci-dessus sont différents, nous ne les distinguerons pas dans ce projet, dans un souci de simplicité.

Nous considérerons donc que la vitesse d'un aéronef sera celle qu'il nous aura transmise dans un message de vitesse en vol, sans faire de distinction entre vitesse sol et vitesse air. De même, nous ne ferons pas de distinction entre route et cap.

Bien entendu, nous devons distinguer les deux sous-types de messages décrits ci-dessus afin de correctement interpréter leur contenu, mais une fois qu'une vitesse et une direction auront été extraits d'un tel message, les deux sous-types seront traités de manière identique.

3. Mise en œuvre Java

3.1. Enregistrement `AirborneVelocityMessage`

L'enregistrement `AirborneVelocityMessage` du sous-paquetage `adsb`, public, représente un message de vitesse en vol du type décrit à la §2.1. Il implémente l'interface `Message` et ses attributs sont :

- `long timeStampNs`, l'horodatage du message, en nanosecondes,
- `IcaoAddress icaoAddress`, l'adresse OACI de l'expéditeur du message,
- `double speed`, la vitesse de l'aéronef, en m/s,
- `double trackOrHeading`, la direction de déplacement de l'aéronef, en radians.

Comme son nom l'indique, l'attribut `trackOrHeading` contient soit la route de l'aéronef, soit son cap. Dans les deux cas, cette valeur est représentée par l'angle – positif et mesuré dans le sens des aiguilles d'une montre – entre le nord et la direction de déplacement de l'aéronef.

`AirborneVelocityMessage` possède un constructeur compact qui lève :

- `NullPointerException` si `icaoAddress` est nul, et
- `IllegalArgumentException` si `timeStampNs`, `speed` ou `trackOrHeading` sont strictement négatifs.

Comme les enregistrements représentant les autres types de message, celui-ci offre une unique méthode statique nommée `of` permettant de construire un message de vitesse en vol à partir d'un message brut :

- `AirborneVelocityMessage of(RawMessage rawMessage)`, qui retourne le message de vitesse en vol correspondant au message brut donné, ou `null` si le sous-type est invalide, ou si la vitesse ou la direction de déplacement ne peuvent pas être déterminés.

3.1.1. Conseils de programmation

Pour déterminer la direction du vecteur vitesse – l'angle qu'il fait avec le nord –, vous pouvez utiliser la méthode `atan2` de `Math`. Lisez bien sa documentation afin de comprendre comment elle fonctionne avant de l'utiliser !

Pour déterminer la norme de ce même vecteur, vous pouvez utiliser la méthode `hypot` de `Math`.

3.2. Classe `MessageParser`

La classe `MessageParser` du sous-paquetage `adsb`, publique et non instanciable, a pour but de transformer les messages ADS-B bruts en messages d'un des trois types décrits précédemment – identification, position en vol, vitesse en vol. Elle n'offre qu'une seule méthode publique (et statique) :

- `Message.parse(RawMessage rawMessage)`, qui retourne l'instance de `AircraftIdentificationMessage`, de `AirbornePositionMessage` ou de `AirborneVelocityMessage` correspondant au message brut donné, ou `null` si le code de type de ce dernier ne correspond à aucun de ces trois types de messages, ou si il est invalide.

Un message brut est invalide si la méthode `of` de la classe correspondant à son code de type retourne `null`.

3.3. Classe `AircraftStateAccumulator`

La classe `AircraftStateAccumulator` du sous-paquetage `adsb`, publique, représente un « accumulateur d'état d'aéronef », c.-à-d. un objet accumulant les messages ADS-B provenant d'un seul aéronef afin de déterminer son état au cours du temps.

Une instance de `AircraftStateAccumulator` est associée à un objet représentant l'état modifiable d'un aéronef, de type `AircraftStateSetter`, qui est passée à son constructeur. Le rôle principal de l'accumulateur est d'appeler les méthodes de modification de cet état afin de le maintenir à jour au fur et à mesure de l'arrivée des messages envoyés par l'aéronef. Par exemple, lorsqu'un message d'identification est envoyé par l'aéronef et transmis à l'accumulateur, il appelle les méthodes `setCategory` et `setCallsign` de l'état afin de lui communiquer ces informations, extraites du message.

`AircraftStateAccumulator` mémorise de plus toujours le dernier message pair et le dernier message impair reçu de l'aéronef, dans le but de pouvoir déterminer sa position.

`AircraftStateAccumulator` est générique, et son paramètre de type, nommé `T` ci-dessous, est borné par `AircraftStateSetter`. Elle possède un unique constructeur

public :

- `AircraftStateAccumulator(T stateSetter)`, qui retourne un accumulateur d'état d'aéronef associé à l'état modifiable donné, ou lève `NullPointerException` si celui-ci est nul.

En plus de ce constructeur, `AircraftStateAccumulator` offre les deux méthodes suivantes :

- `T stateSetter()`, qui retourne l'état modifiable de l'aéronef passé à son constructeur,
- `void update(Message message)`, qui met à jour l'état modifiable en fonction du message donné, en appelant, pour tous les types de message, sa méthode `setLastMessageTimestampNs`, ainsi que :
 - s'il s'agit d'un message d'identification et de catégorie, `setCategory` et `setCallSign`,
 - s'il s'agit d'un message de positionnement en vol, `setAltitude` et, si la position peut être déterminée, `setPosition`,
 - s'il s'agit d'un message de vitesse en vol, `setVelocity` et `setTrackOrHeading`.

Pour que la position puisse être déterminée, il faut que la différence entre l'horodatage du message passé à `update` et celui du dernier message de parité opposée reçu de l'aéronef soit inférieur ou égal à 10 secondes.

3.3.1. Conseils de programmation

La méthode `update` doit déterminer, d'une manière ou d'une autre, le type exact du message qu'on lui passe en argument pour savoir s'il s'agit d'une instance de `AircraftIdentificationMessage`, de `AirbornePositionMessage` ou de `AirborneVelocityMessage`. Plusieurs techniques existent pour faire cela.

La première consiste à utiliser une séquence de tests au moyen de l'opérateur `instanceof`. Cette solution n'est toutefois ni très propre ni très efficace.

La seconde consiste à utiliser le patron de conception Visiteur (*Visitor*), que la plupart d'entre vous a examiné au premier semestre, mais cela est relativement lourd et implique l'existence d'une infrastructure qui n'a pas été mise en place pour les classes implémentant `Message`.

La troisième consiste à utiliser le **filtrage de motif** (*pattern matching*), un concept déjà décrit partiellement à l'étape 1. Il s'agit de la solution la plus propre et la plus efficace, et nous vous conseillons donc fortement son emploi. Elle consiste à utiliser une variante de l'énoncé `switch` qui permet de déterminer de quel classe une valeur est une instance.

L'extrait de programme ci-dessous illustre cette variante de l'énoncé `switch` au moyen

d'une méthode qui prend en argument une valeur de type `Message`, détermine s'il s'agit d'une instance de `AircraftIdentificationMessage`, et dans ce cas, affiche l'indicatif qu'il contient à l'écran ; sinon, un texte par défaut est affiché.

```
void printCallSign(Message m) {  
    switch (m) {  
        case AircraftIdentificationMessage aim ->  
            System.out.println("Indicatif : " + aim.callSign());  
  
        default ->  
            System.out.println("Autre type de message.");  
    }  
}
```

Notez bien que la ligne

```
case AircraftIdentificationMessage aim ->
```

teste d'une part si `m` est une instance de `AircraftIdentificationMessage`, et, si c'est le cas, définit la variable nommée `aim` qui contient ce message et a le type `AircraftIdentificationMessage`. C'est ce qui rend valide l'appel à la méthode `callSign` de `aim` à la ligne suivante.

Pour pouvoir utiliser cette variante du `switch` dans votre projet, il vous faut changer légèrement sa configuration. Pour cela, ouvrez le menu *File*, puis sélectionnez *Project Structure...* Sélectionnez *Project* sur la gauche, puis cliquez sur le menu déroulant à droite de *Language level* et choisissez *17 (Preview) - Pattern matching for switch*.

3.4. Tests

Comme d'habitude, nous ne vous fournissons plus de tests mais [un fichier de vérification de signatures](#) à importer dans votre projet.

Pour tester les classes de cette étape, vous pouvez commencer par écrire une classe implémentant l'interface `AircraftStateSetter` et qui affiche simplement les informations reçues à l'écran. Un squelette d'une telle classe pourrait ressembler à ceci :

```
class AircraftState implements AircraftStateSetter {  
    @Override  
    public void setCallSign(CallSign callSign) {  
        System.out.println("indicatif : " + callSign);  
    }  
  
    @Override  
    public void setPosition(GeoPos position) {  
        System.out.println("position : " + position);  
    }  
}
```

```

}

// ... autres méthodes
}

```

Cela fait, vous pouvez exécuter une version améliorée du programme de test donné à la fin de l'étape 4, qui démodule les 384 messages du fichier d'échantillons fourni et utilise un accumulateur pour suivre l'état de l'aéronef dont l'adresse OACI est [4D2228](#) :

```

public static void main(String[] args) throws IOException {
    String f = "samples_20230304_1442.bin";
    IcaoAddress expectedAddress = new IcaoAddress("4D2228");
    try (InputStream s = new FileInputStream(f)) {
        AdsbDemodulator d = new AdsbDemodulator(s);
        RawMessage m;
        AircraftStateAccumulator<AircraftState> a =
            new AircraftStateAccumulator<>(new AircraftState());
        while ((m = d.nextMessage()) != null) {
            if (!m.icaoAddress().equals(expectedAddress)) continue;

            Message pm = MessageParser.parse(m);
            if (pm != null) a.update(pm);
        }
    }
}

```

qui devrait afficher les mises à jour de l'état suivantes à l'écran :

```

position : (5.620176717638969°, 45.71530147455633°)
position : (5.621292097494006°, 45.715926848351955°)
indicatif : CallSign[string=RYP7JD]
position : (5.62225341796875°, 45.71644593961537°)
position : (5.623420681804419°, 45.71704415604472°)
position : (5.624397089704871°, 45.71759032085538°)
position : (5.625617997720838°, 45.71820789948106°)
position : (5.626741759479046°, 45.718826316297054°)
position : (5.627952609211206°, 45.71946484968066°)
position : (5.629119873046875°, 45.72007002308965°)
position : (5.630081193521619°, 45.7205820735544°)
position : (5.631163045763969°, 45.72120669297874°)
indicatif : CallSign[string=RYP7JD]
position : (5.633909627795219°, 45.722671514377°)
position : (5.634819064289331°, 45.72314249351621°)

```


4. Résumé

Pour cette étape, vous devez :

- écrire les classes `AirborneVelocityMessage`, `MessageParser` et `AircraftStateAccumulator` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies,
- rendre votre code au plus tard le **31 mars 2023 à 18h00**, au moyen du programme `Submit.java` fourni et des jetons disponibles sur votre [page privée](#).

Ce rendu est un rendu testé, auquel 18 points sont attribués, au prorata des tests unitaires passés avec succès.

N'attendez surtout pas le dernier moment pour effectuer votre rendu, car vous n'êtes pas à l'abri d'imprévus. **Souvenez-vous qu'aucun retard, aussi insignifiant soit-il, ne sera toléré !**